

Quantization as a Solution to Model Size

Research Report — Task 0

Case studies: BERT and LLaMA

1. Introduction: The Problem of Model Size

Models such as BERT (110M-340M parameters) and LLaMA (7B-70B parameters) achieve state-of-the-art results across natural language processing tasks, but their size creates serious practical problems for storage, deployment, and inference:

- Memory footprint - LLaMA-7B stored in 32-bit floating point (fp32) needs roughly 7 billion parameters x 4 bytes $\approx$ 28 GB of RAM/VRAM just to hold the weights.
- Inference latency - larger tensors mean more memory bandwidth is needed to move weights in and out of compute units, which is often the real bottleneck (not raw FLOPs).
- Deployment cost - running such models needs expensive, high-memory GPUs (A100/H100), which is unrealistic for edge devices, laptops, or many production budgets.
- Energy consumption / carbon footprint - bigger models draw more power both to train and to serve.

1.1 Common Strategies to Address Large Model Size

Strategy	Idea	Trade-off
Quantization	Represent weights/activations with fewer bits (e.g., int8, int4)	Small accuracy loss, large memory/speed gain
Pruning	Remove redundant weights, neurons, or attention heads	Needs careful fine-tuning to avoid accuracy drop
Knowledge Distillation	Train a small 'student' model to mimic a large 'teacher' (e.g., DistilBERT)	Requires retraining; still a separate smaller model
LoRA / Adapters	Add small trainable matrices instead of updating all weights	Mainly helps fine-tuning cost, not base inference size
Efficient architectures	Design smaller/faster architectures from scratch (e.g., MobileBERT, TinyLlama)	Requires new pretraining

This report focuses on quantization, since it is the most broadly applicable technique: it can be applied after training (no retraining needed), works on virtually any architecture, and typically gives a 2x-4x memory reduction with minimal accuracy loss.

2. What Is Quantization?

Quantization is the process of mapping a large (often continuous or high-precision) set of values to a smaller, discrete set of values, using fewer bits per number. Neural network weights and activations are normally stored as 32-bit floating point numbers (fp32). Quantization converts them into lower-precision representations such as 16-bit float (fp16/bf16), 8-bit integer (int8), or even 4-bit integer (int4).

2.1 The Mathematics of Quantization

The most common scheme is affine (uniform) quantization. A real value x is mapped to an integer x_q using a scale s and a zero-point z :

$$x_q = \text{round}(x/s) + z, \quad x_q \in [q_{\min}, q_{\max}]$$

and it can be approximately recovered ("dequantized") via:

$$\hat{x} = s \cdot (x_q - z)$$

For symmetric quantization (commonly used for weights), $z = 0$ and the scale is derived from the maximum absolute value in the tensor:

$$s = \max(|x|) / (2^{(b-1)} - 1)$$

where b is the number of bits (e.g., $b = 8 \Rightarrow$ values in $[-127, 127]$).

The quantization error for a single value is bounded by half the step size:

$$|x - \hat{x}| \leq s/2$$

Because neural networks are highly overparameterized and somewhat robust to noise, this small, structured rounding error usually causes only a minor drop in accuracy, while the memory savings are large and guaranteed:

$$\text{Memory}(b\text{-bit}) = \text{Memory}(32\text{-bit}) \times (b/32)$$

So int8 gives a theoretical 4x reduction and int4 gives an 8x reduction versus fp32.

2.2 Types of Quantization

By when quantization is applied

- Post-Training Quantization (PTQ) - the model is trained normally in full precision, and quantization is applied afterward, with little or no retraining. Fast and easy, with only a small risk of accuracy loss.
- Dynamic quantization: weights are quantized ahead of time; activations are quantized on-the-fly during inference, per batch. Simple, no calibration data needed. Works well for models bottlenecked by memory-bound ops (e.g., Linear/LSTM layers) - exactly the case for Transformers.
- Static quantization: both weights and activations are quantized ahead of time. Requires a calibration dataset to estimate activation ranges. Faster at runtime than dynamic quantization, but needs more setup.

- Quantization-Aware Training (QAT) - fake-quantization operations are inserted during training/fine-tuning, so the model learns weights that are robust to quantization noise. Gives the best accuracy at low bit-widths, but requires access to training data and compute.

By what is quantized

- Weight-only quantization - only the weight matrices are compressed (e.g., int4 weights with fp16 activations). This is the dominant approach for large LLMs (GPTQ, AWQ, bitsandbytes NF4), since weights dominate memory and activations are more sensitive to precision loss.
- Weight + activation quantization - both are quantized (e.g., standard int8 PTQ/QAT), giving extra speedups on hardware with native int8 kernels, at slightly higher risk to accuracy.

By bit-width

- fp16 / bf16 (16-bit) - "half precision"; minimal accuracy loss; 2x smaller than fp32.
- int8 (8-bit) - e.g., LLM.int8() (Dettmers et al., 2022); 4x smaller than fp32.
- int4 / NF4 (4-bit) - e.g., QLoRA's NormalFloat4 (Dettmers et al., 2023), GPTQ, AWQ; 8x smaller than fp32 - the current sweet spot for running 7B+ LLMs on consumer GPUs.

2.3 The Technique Chosen for This Report

For the hands-on sections below we use the same technique for both models: 8-bit weight quantization via bitsandbytes' LLM.int8(), applied as Post-Training Quantization (no retraining or calibration data needed).

We chose this technique because:

- It requires no retraining or calibration dataset - it is applied with a single flag (load_in_8bit=True / load_in_4bit=True) when loading a Hugging Face model.
- It is directly supported in the transformers + bitsandbytes ecosystem, so it runs out-of-the-box on a free Google Colab GPU.
- It scales from small models (BERT) to very large ones (LLaMA), letting us demonstrate the same idea on both models in this report.

3. Case Study A - Quantizing BERT (8-bit, LLM.int8())

The cell below loads BERT once in full precision (fp32) and once with bitsandbytes 8-bit quantization, then simply compares the resulting model sizes.

```
!pip install -q -U transformers accelerate bitsandbytes

import torch
from transformers import AutoModelForSequenceClassification, BitsAndBytesConfig

MODEL_NAME = "bert-base-uncased"
size_mb = lambda m: sum(p.nelement()*p.element_size() for p in m.parameters()) / 1024**2

# Baseline: full-precision (fp32)
model_fp32 = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
print(f"fp32 size: {size_mb(model_fp32):.2f} MB")

# Quantized: 8-bit (LLM.int8())
model_int8 = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2,
    quantization_config=BitsAndBytesConfig(load_in_8bit=True),
    device_map="auto",
)
print(f"int8 size: {size_mb(model_int8):.2f} MB")
```

Results

Metric	fp32 (baseline)	int8 (LLM.int8())	Improvement
Model size (MB)	417.65	172.96	~2.41x smaller

4. Case Study B - Quantizing LLaMA (8-bit, LLM.int8())

Because official meta-llama weights are gated, this example uses openlm-research/open_llama_3b, an openly licensed LLaMA-architecture model, so it runs on Colab without extra authentication. The same 8-bit technique used for BERT above is applied here, to keep the comparison consistent.

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

MODEL_NAME = "openlm-research/open_llama_3b"
size_gb = lambda m: sum(p.nelement()*p.element_size() for p in m.parameters()) / 1024**3

# Baseline: half precision (fp16)
model_fp16 = AutoModelForCausalLM.from_pretrained(MODEL_NAME, torch_dtype=torch.float16,
device_map="auto")
print(f"fp16 size: {size_gb(model_fp16):.2f} GB")

# Quantized: 8-bit (LLM.int8())
model_int8 = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    quantization_config=BitsAndBytesConfig(load_in_8bit=True),
    device_map="auto",
)
print(f"int8 size: {size_gb(model_int8):.2f} GB")
```

Results

Metric	fp16 (baseline)	int8 (LLM.int8())	Improvement
Model size (GB)	6.38	3.38	~1.89x smaller

5. Summary

	BERT (~110M params)	LLaMA (3B params)
Quantization used	8-bit (LLM.int8())	8-bit (LLM.int8())
Measured size before -> after	417.65 MB -> 172.96 MB	6.38 GB -> 3.38 GB
Observed reduction	~2.41x	~1.89x
Retraining required?	No (PTQ)	No (PTQ)
Main benefit	Smaller footprint, easier deployment	Makes multi-billion-parameter models fit on a single consumer/Colab GPU

Key takeaway: Quantization is one of the most effective, low-effort ways to shrink models like BERT and LLaMA. Because it can be applied after training with just a couple of lines of code via transformers and bitsandbytes, it offers a very high savings-per-effort ratio compared to alternatives like pruning, distillation, or training smaller architectures from scratch - which explains why it is the default technique used to serve today's large language models efficiently.